
LAPORAN PRAKTIKUM
MATA KULIAH TEKNOLOGI IOT II

ARSITEKTUR INTERNET OF THINGS DAN MIKROKONTROLER



Oleh:

Nama: Aila Nadianti

NPM : 25782059

Kelas : 3C

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

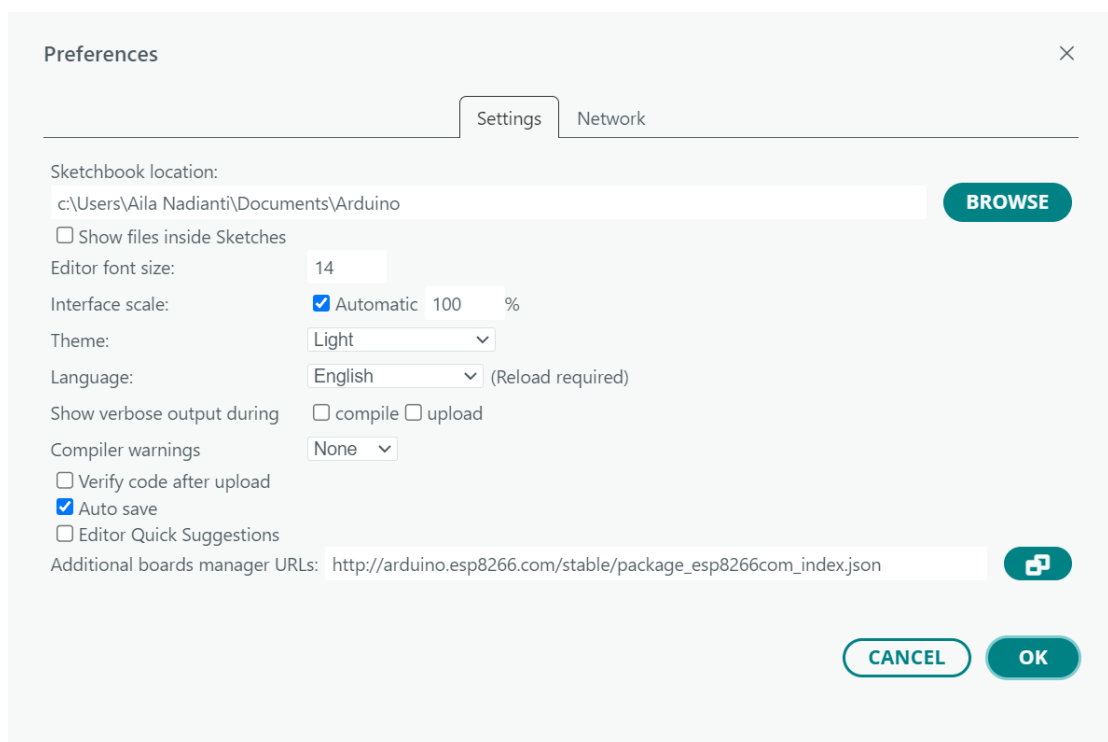
1.4.2 Konfigurasi dan Instalasi Lingkungan Pengembangan (Arduino IDE)

Arduino IDE merupakan perangkat lunak yang digunakan untuk menulis, mengompilasi, dan mengunggah program ke mikrokontroler. Sebelum digunakan untuk pemrograman, Arduino IDE perlu diinstal dan dikonfigurasi terlebih dahulu sesuai dengan board yang akan digunakan. Proses instalasi dimulai dengan mengunduh Arduino IDE melalui situs resmi Arduino sesuai dengan sistem operasi yang digunakan. Setelah file instalasi selesai diunduh, jalankan installer dengan melakukan *double click*. Selanjutnya, ikuti tahapan instalasi yang tersedia, seperti menyetujui ketentuan lisensi, menentukan komponen yang diperlukan, memilih lokasi penyimpanan, kemudian menekan tombol **Install**. Setelah proses instalasi selesai, Arduino IDE dapat dijalankan melalui **Start Menu** maupun *shortcut* pada desktop.

Konfigurasi Board ESP8266

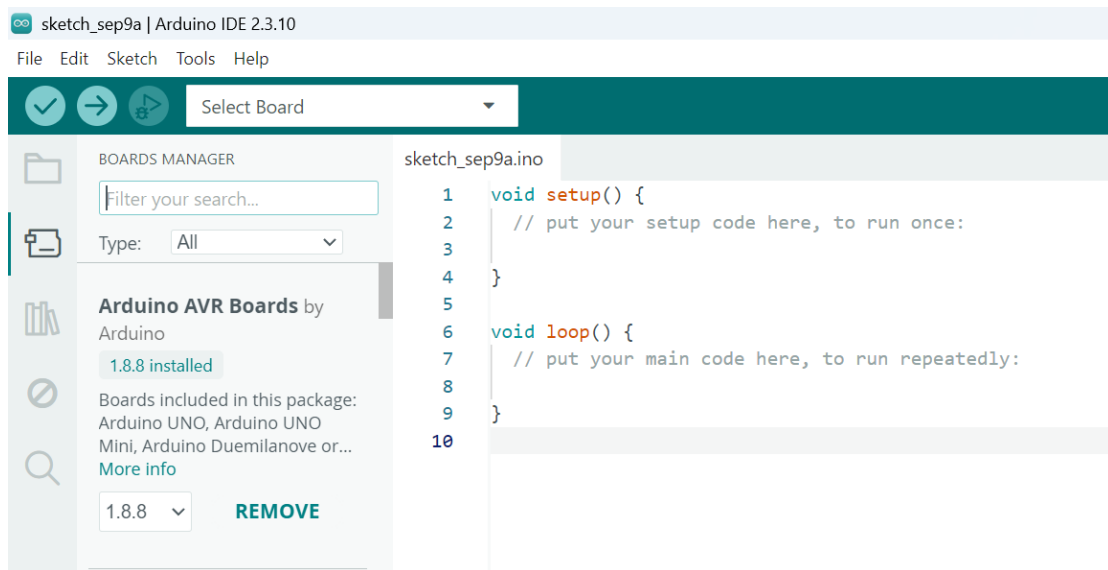
Agar Arduino IDE dapat mengenali dan digunakan untuk memprogram mikrokontroler ESP8266, diperlukan instalasi *board package* ESP8266. Konfigurasi dilakukan dengan langkah-langkah berikut:

1. Jalankan aplikasi **Arduino IDE**.
2. Pilih menu **File > Preferences**.
3. Pada bagian **Additional Boards Manager URLs**, masukkan alamat berikut:
http://arduino.esp8266.com/stable/package_esp8266com_index.json



4. Setelah itu, masuk ke menu **Tools > Board > Boards Manager....**
5. Cari **esp8266** pada kolom pencarian.

6. Pilih paket **esp8266 by ESP8266 Community**, kemudian klik **Install**.
7. Tunggu hingga proses instalasi paket selesai.



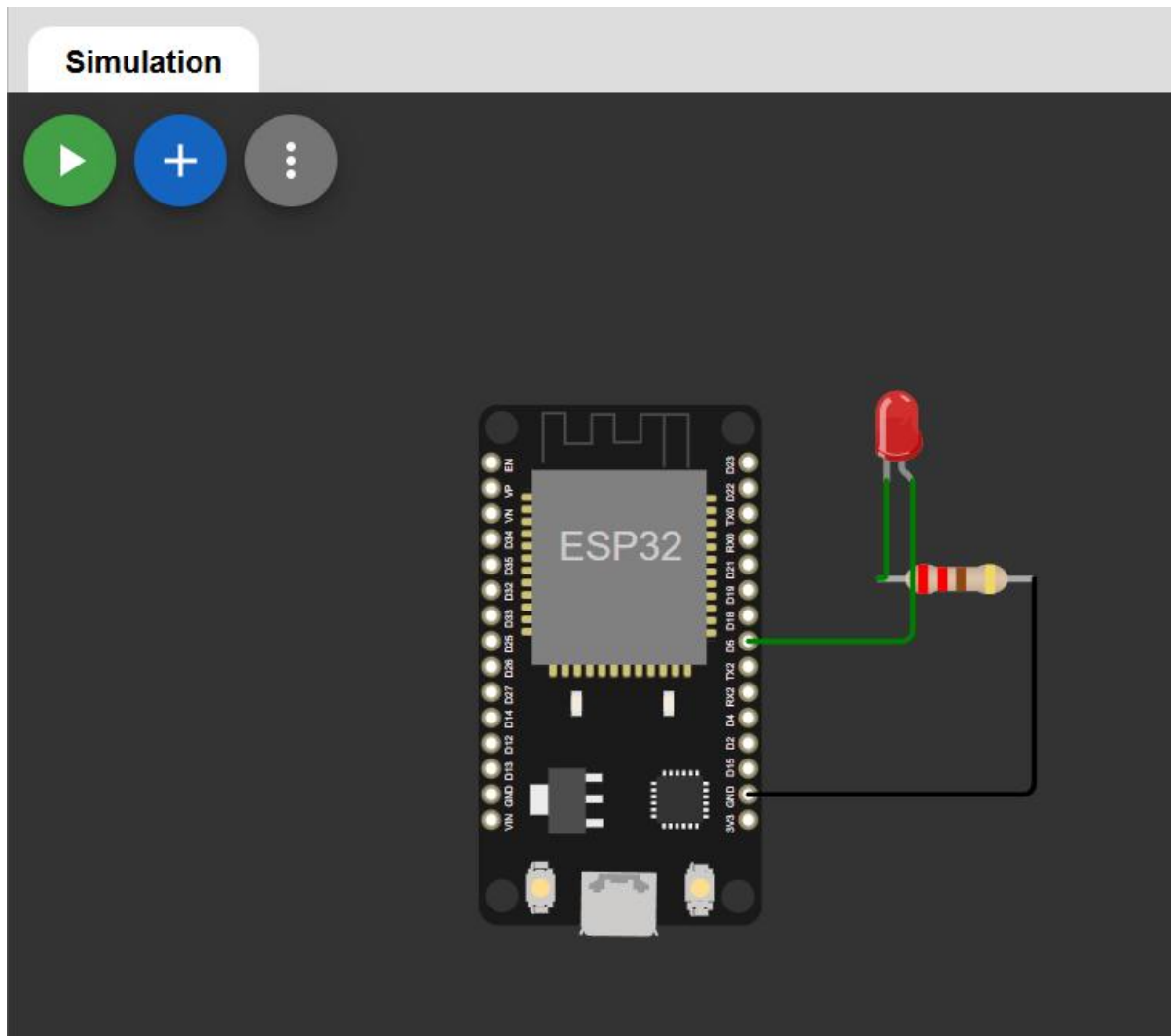
Setelah *board package* berhasil dipasang, lakukan pengaturan board melalui menu **Tools**. Pada bagian **Board**, pilih **NodeMCU 1.0 (ESP-12E Module)**. Kemudian, hubungkan NodeMCU ke laptop menggunakan kabel USB. Setelah perangkat terdeteksi, pilih **Port COM** yang sesuai dengan port yang muncul pada Arduino IDE.

Dengan konfigurasi tersebut, Arduino IDE sudah siap digunakan untuk membuat dan mengunggah program ke board NodeMCU ESP8266.

1.5 Praktikum

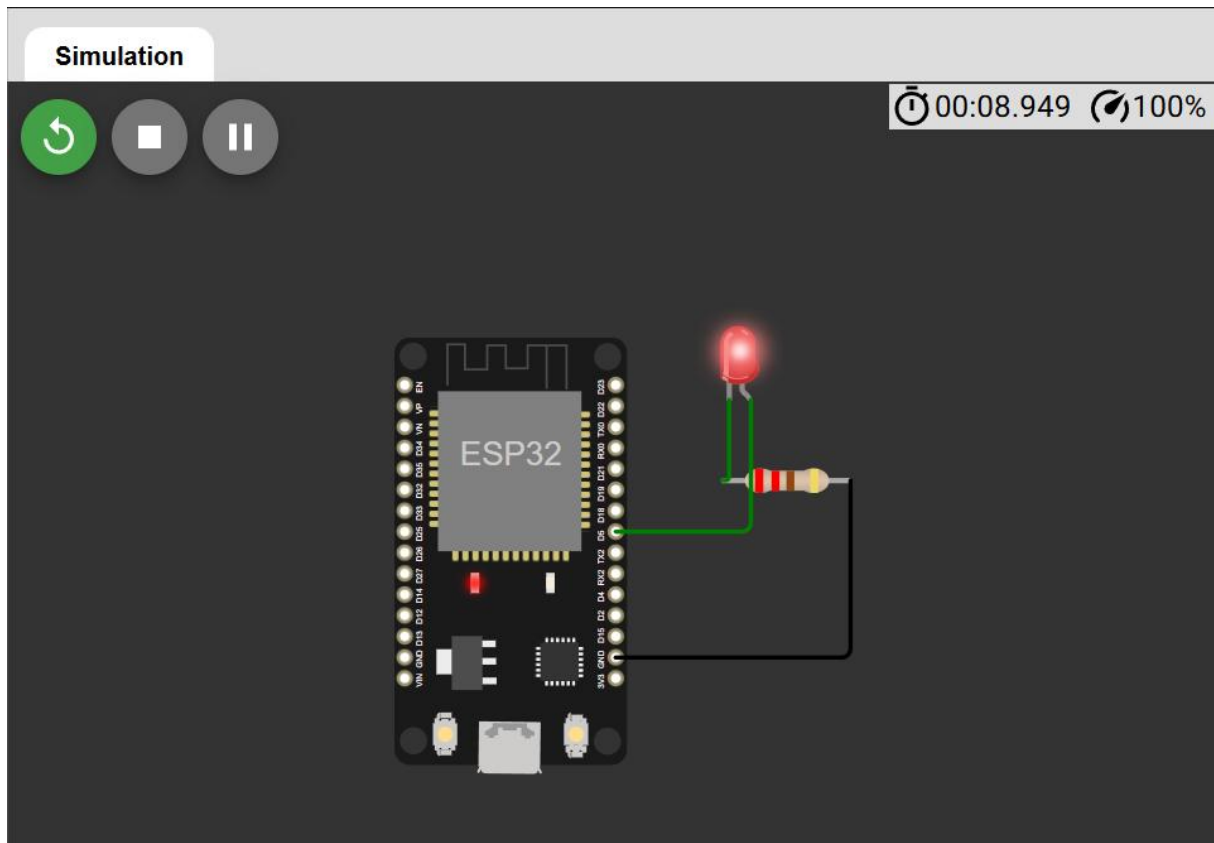
1.5.1 Praktikum 1: Digital Output (Menyalakan LED)

Praktikum ini bertujuan memprogram NodeMCU agar mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal.



Gambar 1. Simulasi rangkaian Digital Output

- a. Pengendali Utama: Board mikrokontroler mengelola eksekusi algoritma dan pendistribusian sinyal keluaran.
- b. Umpan Balik Visual: LED menampilkan status kerja; bernilai HIGH saat menyala dan LOW saat padam.
- c. Proteksi Arus: Resistor berfungsi menahan arus berlebih agar komponen LED tidak terbakar.
- d. Konektivitas Sinyal: Jalur output digital dialokasikan pada pin D1 (GPIO5) untuk mengendalikan LED.
- e. Referensi Ground: Terminal GND menyediakan jalur return arus dan menyamakan acuan ground sistem.



Gambar 2. Kondisi LED saat program dijalankan

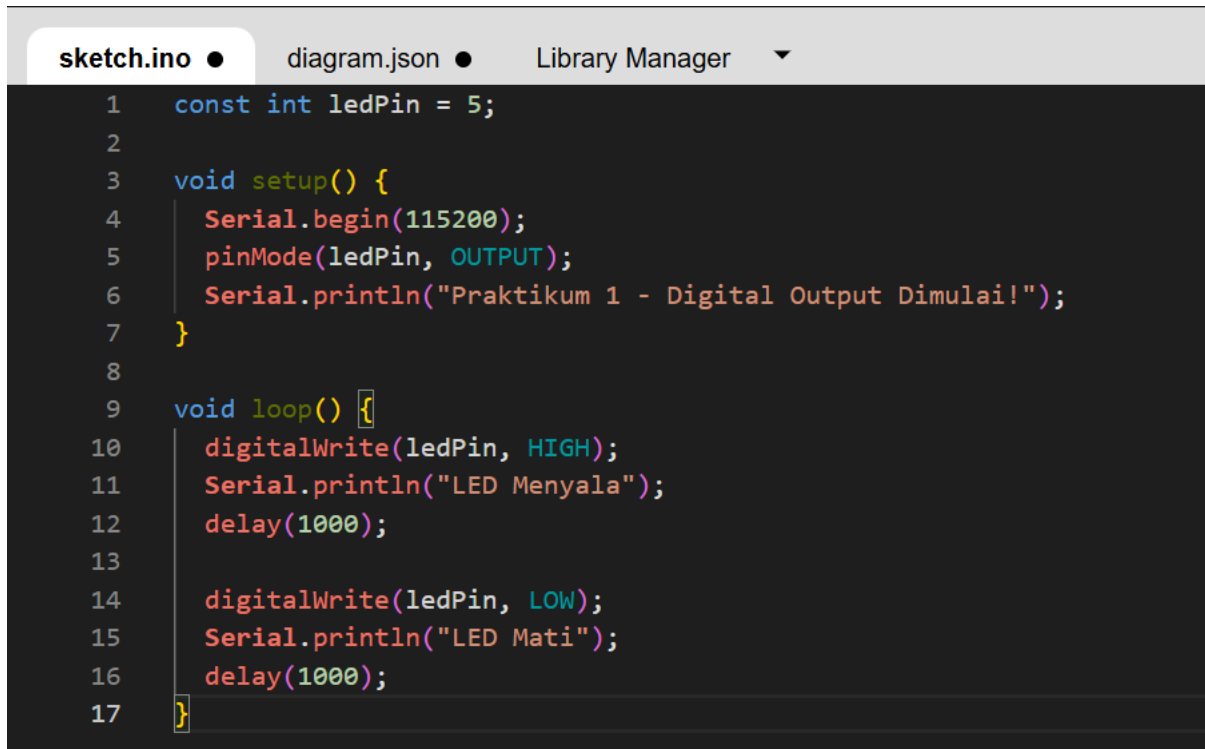
Tampilan *screenshot* menunjukkan kondisi LED yang sedang menyala akibat penerimaan nilai logika HIGH dari pin keluaran mikrokontroler. Begitu instruksi program berubah menjadi LOW, arus terhenti dan LED otomatis padam. Mekanisme transmisi sinyal HIGH-LOW ini menjadi dasar utama dalam pengontrolan komponen *actuator* sederhana.

```
Praktikum 1 - Digital Output Dimulai!
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
```

Gambar 3. Tampilan Serial Monitor Digital Output

Fungsi utama Serial Monitor adalah menampilkan keluaran informasi dari skrip yang sedang dieksekusi mikrokontroler. Dalam pengujian ini, Serial Monitor mencetak pembaruan kondisi

LED secara bergantian (saat menyala dan padam). Tampilan log tersebut menjadi sarana validasi bahwa instruksi program berhasil dijalankan dan transisi logika terjadi sesuai perintah.

The image shows a screenshot of the Arduino IDE interface. At the top, there are three tabs: 'sketch.ino' (selected), 'diagram.json', and 'Library Manager'. The main area displays the code for 'sketch.ino'. The code is as follows:

```
1  const int ledPin = 5;
2
3  void setup() {
4      Serial.begin(115200);
5      pinMode(ledPin, OUTPUT);
6      Serial.println("Praktikum 1 - Digital Output Dimulai!");
7  }
8
9  void loop() {
10     digitalWrite(ledPin, HIGH);
11     Serial.println("LED Menyala");
12     delay(1000);
13
14     digitalWrite(ledPin, LOW);
15     Serial.println("LED Mati");
16     delay(1000);
17 }
```

Gambar 4. Kode program sketch.ino

File sketch.ino merupakan bagian yang berisi program untuk mengontrol LED. Program menentukan pin LED sebagai OUTPUT, kemudian memberikan HIGH untuk menyalakan LED dan LOW untuk mematikannya. Perintah delay(1000) membuat setiap kondisi berlangsung sekitar satu detik sehingga LED terlihat berkedip secara teratur.

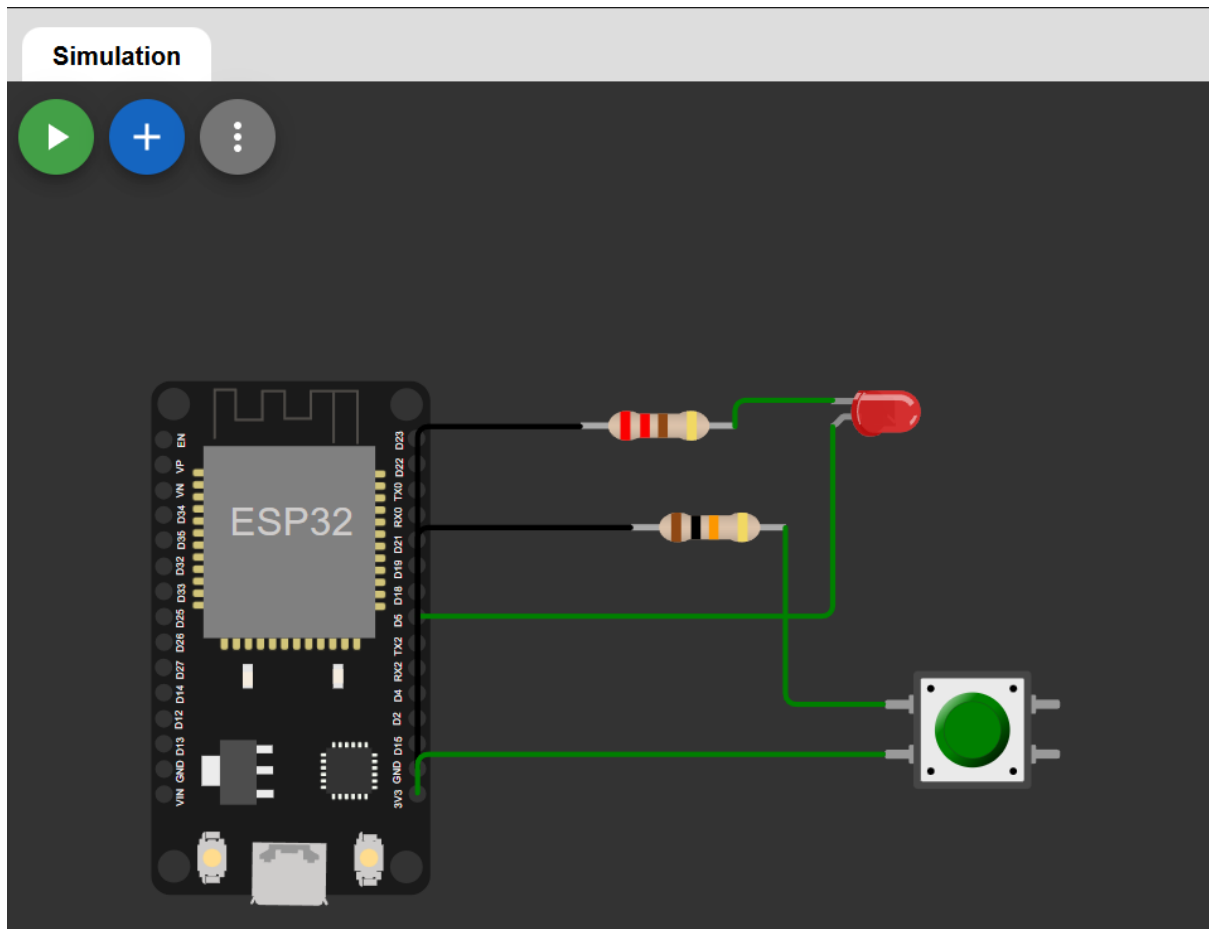
```
1 {
2   "version": 1,
3   "author": "Praktikum 1.5.1 ESP32",
4   "editor": "wokwi",
5   "parts": [
6     { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
7     {
8       "type": "wokwi-led",
9       "id": "led1",
10      "top": -100,
11      "left": 150,
12      "attrs": { "color": "red" }
13    },
14    {
15      "type": "wokwi-resistor",
16      "id": "r1",
17      "top": 20,
18      "left": 150,
19      "attrs": { "value": "220" }
20    }
21  ],
22  "connections": [
23    [ "esp:D23", "led1:A", "green", [ "h0" ] ],
24    [ "led1:C", "r1:1", "green", [ "v0" ] ],
25    [ "r1:2", "esp:GND.1", "black", [ "v0" ] ]
26  ],
27  "dependencies": {}
28 }
```

Gambar 5. Konfigurasi diagram.json

Diagram.json itu ibarat denah atau gambar rangkaiannya—di situ didaftarkan semua alat yang dipakai dalam simulasi, mulai dari board mikrokontroler, LED, resistor, sampai kabel-kabel yang menyambungkan antar pin-nya. Jadi sederhananya, kodingan di sketch.ino itu bertindak sebagai "otak" yang ngatur cara kerja rangkaian, sedangkan diagram.json yang nyediain fisik perangkat dan susunannya.

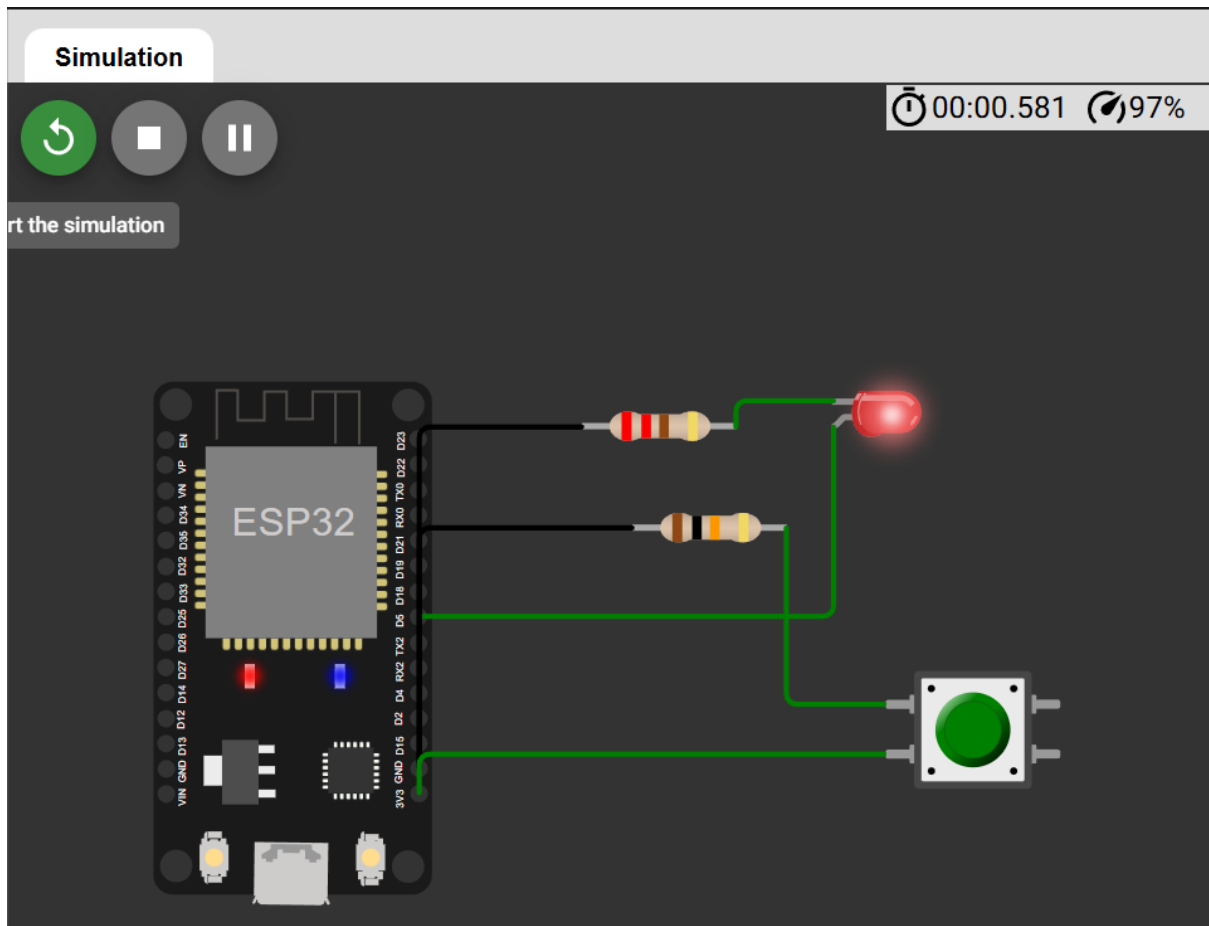
1.5.2 Praktikum 2: Digital Input (Membaca Push Button)

Praktikum ini bertujuan mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor pull-down eksternal.



Gambar 6. Simulasi rangkaian Digital Input

- Push button berfungsi sebagai input yang memberikan perubahan kondisi ketika ditekan.
- Pin **D4/GPIO4** terhubung ke tombol (di antara push button dan resistor pull-down) untuk membaca status input tombol. Resistor pull-down 10kΩ menjaga input tetap LOW ketika tombol tidak ditekan.
- Resistor pull-down 10kΩ (terhubung ke GND via pin **D21/GPIO21**) menjaga input tetap LOW ketika tombol tidak ditekan. Saat tombol ditekan, input berubah menjadi HIGH sehingga program dapat meresponsnya.
- LED merah terhubung ke pin **D5/GPIO5** (dengan pembatas arus resistor 220Ω yang terhubung ke GND via pin **D23/GPIO23**) sebagai indikator hasil pembacaan tombol.
- Daya 3.3V diambil dari pin **3V3** menuju salah satu kaki push button, sehingga saat tombol ditekan, input pada pin D4 berubah menjadi HIGH.



Gambar 7. Kondisi rangkaian saat tombol ditekan

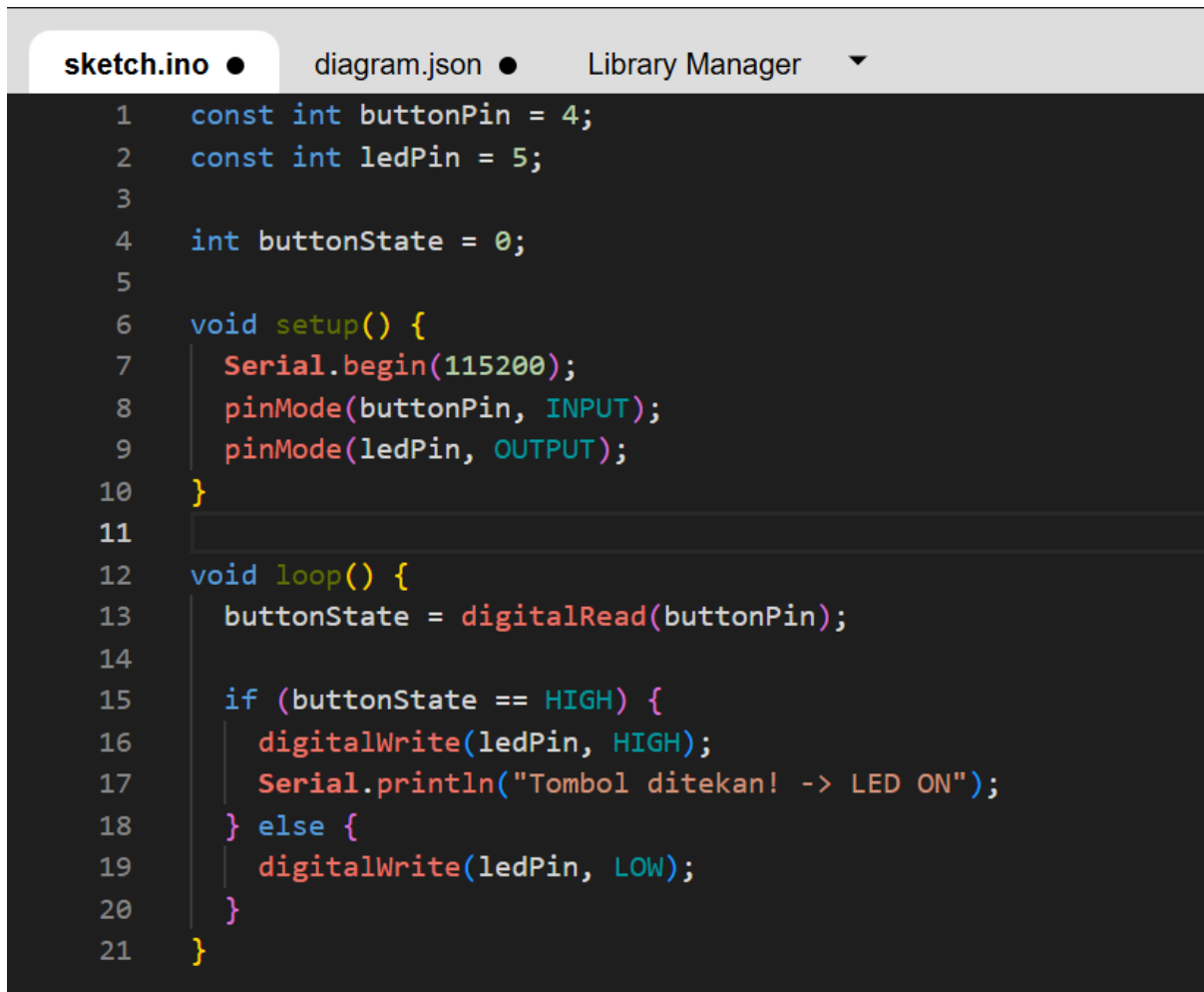
Saat tombol ditekan, arus listrik 3,3V mengalir ke pin D4/GPIO4 sehingga sistem membacanya sebagai sinyal HIGH. Kondisi ini memicu program untuk menyalakan LED sebagai respon visual. Ketika tombol dilepas, jalur 3,3V terputus dan resistor pull-down menarik pin D4 kembali ke posisi LOW (GND), yang menyebabkan LED otomatis padam.

```
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
```

Gambar 8. Tampilan Serial Monitor saat tombol ditekan

Di Serial Monitor, muncul notifikasi kalau tombol lagi ditekan dan posisi LED dalam kondisi ON. Pesan ini bisa keluar gara-gara program berhasil ngebaca status input-nya pakai perintah `digitalRead()`.

Karena di praktikum ini sifat tombolnya masih *momentary*, LED-nya cuma bakal nyala pas tombolnya ditahan aja. Begitu tombolnya dilepas, LED-nya otomatis mati lagi.

The image shows a screenshot of the Arduino IDE interface. At the top, there are three tabs: 'sketch.ino' (selected), 'diagram.json', and 'Library Manager'. The main area displays a C++ sketch for controlling an LED with a button. The code is as follows:

```
1  const int buttonPin = 4;
2  const int ledPin = 5;
3
4  int buttonState = 0;
5
6  void setup() {
7      Serial.begin(115200);
8      pinMode(buttonPin, INPUT);
9      pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13     buttonState = digitalRead(buttonPin);
14
15     if (buttonState == HIGH) {
16         digitalWrite(ledPin, HIGH);
17         Serial.println("Tombol ditekan! -> LED ON");
18     } else {
19         digitalWrite(ledPin, LOW);
20     }
21 }
```

Gambar 9. Kode program sketch.ino Digital Input

Kode program pada sketch.ino diawali dengan mengatur pin tombol sebagai input digital dan pin LED sebagai output. Melalui fungsi `digitalRead(buttonPin)`, program memeriksa status tombol secara berkala. Ketika tombol terdeteksi bernilai **HIGH**, perintah `digitalWrite(ledPin, HIGH)` dieksekusi untuk menyalakan LED sekaligus mengirimkan notifikasi pada Serial Monitor. Jika tidak ditekan (**LOW**), LED langsung dimatikan melalui `digitalWrite(ledPin, LOW)`.

```
sketch.ino • diagram.json • Library Manager

1 {
2   "version": 1,
3   "author": "Praktikum 1.5.2 ESP32",
4   "editor": "wokwi",
5   "parts": [
6     { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
7     {
8       "type": "wokwi-led",
9       "id": "led1",
10      "top": -10.8,
11      "left": 256.2,
12      "rotate": 90,
13      "attrs": { "color": "red" }
14    },
15    {
16      "type": "wokwi-resistor",
17      "id": "r_led",
18      "top": 13.55,
19      "left": 163.2,
20      "attrs": { "value": "220" }
21    },
22    {
23      "type": "wokwi-resistor",
24      "id": "r_pull",
25      "top": 51.95,
26      "left": 182.4,
27      "attrs": { "value": "10000" }
28    },
29    {
30      "type": "wokwi-pushbutton",
31      "id": "btn1",
32      "top": 111.8,
33      "left": 278.4,
34      "attrs": { "color": "green" }
35    }
36  ],
37  "connections": [
38    [ "esp:D5", "led1:A", "green", [] ],
39    [ "led1:C", "r_led:2", "green", [] ],
40    [ "r_led:1", "esp:GND.1", "black", [] ],
41    [ "r_pull:1", "esp:GND.1", "black", [] ],
42    [ "esp:D4", "btn1:1.L", "green", [] ],
43    [ "btn1:1.L", "r_pull:2", "green", [] ],
44    [ "esp:D2", "btn1:2.L", "green", [] ],
45    [ "r_pull:2", "btn1:1.1", "green", [ "v0" ] ],
46    [ "btn1:2.1", "esp:3V3", "green", [ "h0" ] ]
47  ],
48  "dependencies": {}
49 }
```

Gambar 10. Bagian awal konfigurasi diagram.json

Diagram rangkaian terdiri dari board ESP32 (wokwi-esp32-devkit-v1), LED merah, resistor 220 ohm untuk LED, resistor 10k ohm untuk *pull-down*, *push button* hijau, dan kabel-kabel yang saling menghubungkan antar pin-nya. Semua komponen ini dirangkai sesuai konsep *input pull-down* yang dipakai di praktikum ini.

1.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda. Jelaskan hasilnya di dalam laporan!

1. Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / pull-down), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut tepat pada detik di mana NodeMCU baru diberi daya?

Jawab: Pin D8 (GPIO 15) pada ESP8266/NodeMCU merupakan *strapping pin* yang menentukan *boot mode*. GPIO 15 dipersyaratkan harus bernilai LOW saat proses *booting* berlangsung agar ESP8266 dapat booting secara normal dari flash memory.

Dampak saat tombol ditekan tepat saat booting: Jika tombol berkonfigurasi *pull-down* (Active-High) ditekan saat daya baru diberikan, arus 3,3V akan mengalir ke pin D8 sehingga kondisinya menjadi HIGH pada detik pertama booting. Hal ini menyebabkan NodeMCU gagal masuk ke mode operasi normal (*normal boot failure* / mengalami *boot loop*) dan program tidak akan berjalan.

2. Latihan 2 (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara tidak simetris, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.

Jawab: `delay(200); // Durasi LED menyala cepat (200 ms)`
`delay(800); // Durasi LED mati lebih lama (800 ms)`

3. Latihan 3 (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)?

Apakah program akan mengalami error kompilasi?

Tidak. Sisi perangkat lunak/kode tetap valid dan berhasil dikompilasi karena error kompilasi (*syntax error*) hanya terjadi jika ada kesalahan penulisan tata bahasa kode.

Mengapa LED tidak menyala?

LED (*Light Emitting Diode*) adalah komponen dioda yang bersifat *polar* (memiliki arah polaritas) dan hanya mengalirkan arus secara searah (*forward bias*). Jika Anoda dihubungkan ke GND dan Katoda ke D1 (HIGH), LED berada dalam kondisi prabebas mundur (*reverse bias*) sehingga menahan aliran arus listrik dan LED tidak dapat menyala.

4. Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru mati."

Jawab:

```
if (buttonState == LOW) {  
    digitalWrite(ledPin, HIGH); // LED menyala saat tombol TIDAK ditekan  
    Serial.println("Tombol tidak ditekan -> LED ON");  
} else {  
    digitalWrite(ledPin, LOW); // LED mati saat tombol DITEKAN  
}
```

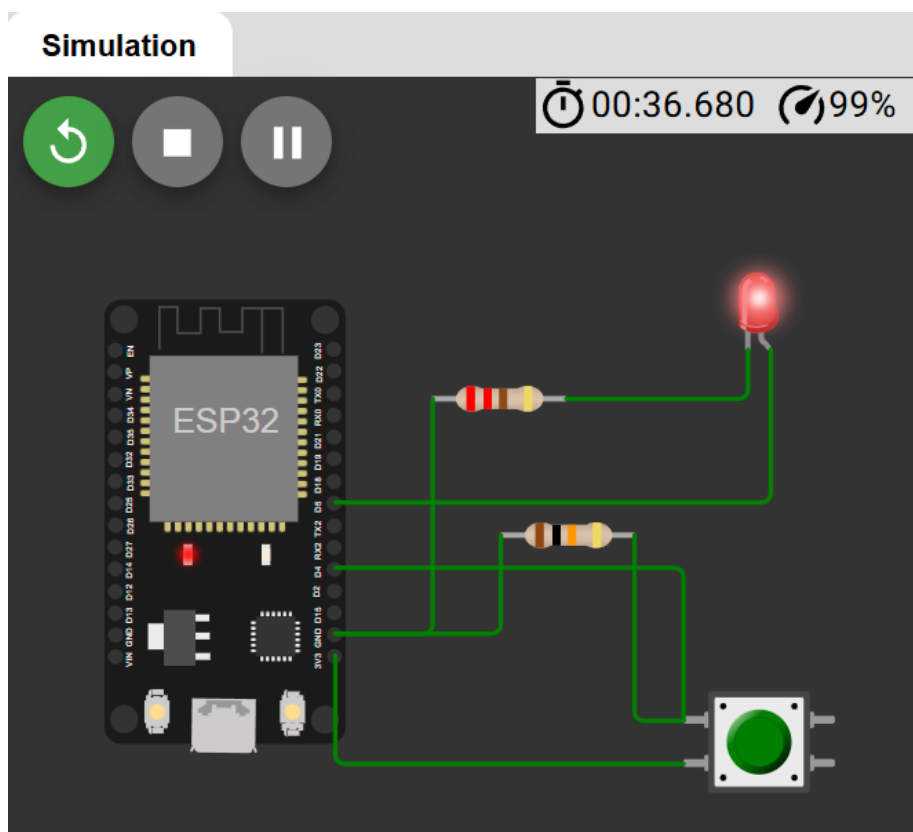
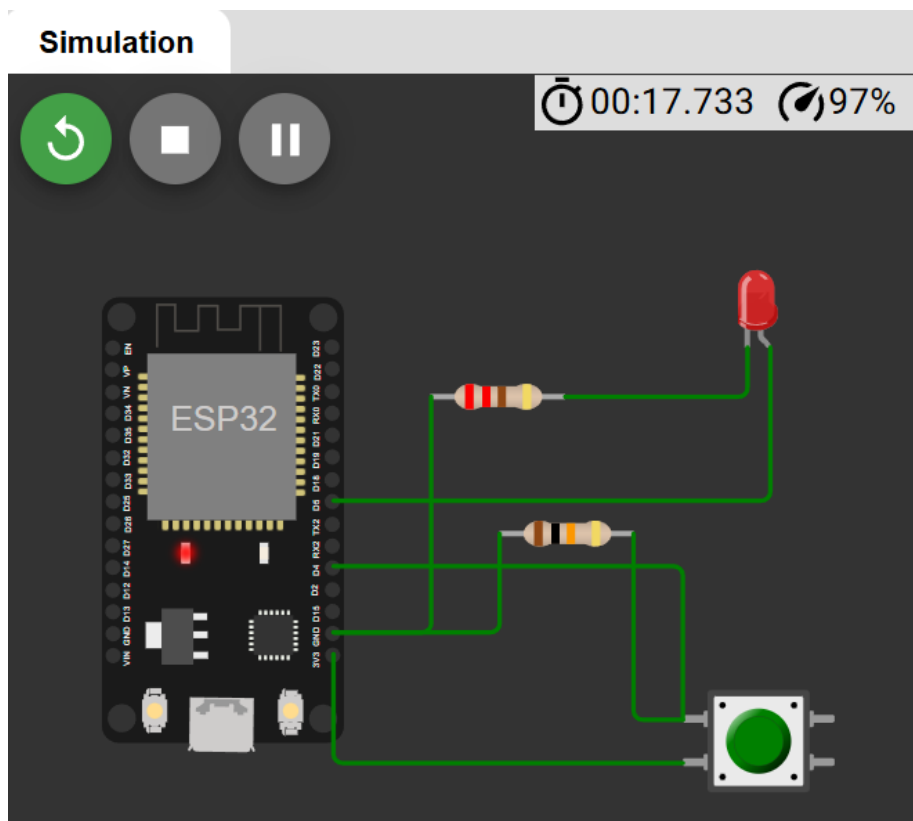
1.7 Tugas

Tugas ini harus dikerjakan secara mandiri untuk mengukur kemampuan integrasi dan logika pemrograman Anda.

Modifikasi Sistem Sakelar Toggle (Latching)

- Tujuan: Mampu mengimplementasikan logika pelacakan status (state tracking) dan membedakan antara perilaku tombol momentary dengan latching.
- Tingkat Kesulitan: Menengah
- Instruksi:
 - a. Gunakan konfigurasi hardware yang persis sama dengan Praktikum 2 (Rangkaian Pull-Down).
 - b. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
 - Jika tombol ditekan 1 kali lalu dilepas, LED menyala dan tetap menyala.
 - Jika tombol ditekan 1 kali lagi lalu dilepas, LED mati dan tetap mati.
 - c. Petunjuk: Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (debouncing pada tombol fisik).
- Kriteria Keberhasilan:
 - a. Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan satu kali tekanan singkat (bukan ditahan).
 - b. Tidak ada fenomena flickering (lampu berkedip acak atau gagal mati) saat tombol ditekan.

- c. Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan state dan debounce bekerja.



Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF

sketch.ino • diagram.json • Library Manager ▼

```
1  const int ledPin = 5;      // LED terhubung ke GPIO 23
2  const int buttonPin = 4;   // Push button terhubung ke GPIO 22
3
4  bool ledState = false;
5
6  void setup() {
7      Serial.begin(115200);
8
9      pinMode(ledPin, OUTPUT);
10     pinMode(buttonPin, INPUT);
11
12     // Kondisi awal LED mati
13     digitalWrite(ledPin, LOW);
14
15     Serial.println("=== SISTEM SAKELAR TOGGLE ESP32 ===");
16     Serial.println("LED awal: OFF");
17 }
18
19 void loop() {
20     int buttonState = digitalRead(buttonPin);
21
22     // Jika tombol ditekan
23     if (buttonState == HIGH) {
24
25         // Mengubah kondisi LED
26         ledState = !ledState;
27
28         digitalWrite(ledPin, ledState);
29
30         if (ledState) {
31             Serial.println("Tombol ditekan -> LED ON");
32         } else {
33             Serial.println("Tombol ditekan -> LED OFF");
34         }
35
36         // Debounce untuk mencegah satu tekanan
37         // terbaca berkali-kali
38         delay(200);
39
40         // Menunggu tombol dilepas
41         while (digitalRead(buttonPin) == HIGH) {
42             delay(10);
43         }
44     }
45 }
```

```
1 {
2   "version": 1,
3   "author": "Tugas 1.7 ESP32 - Toggle Latching Switch",
4   "editor": "wokwi",
5   "parts": [
6     { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": -120.1, "left": -24.2, "attrs": {} },
7     { "type": "wokwi-led", "id": "led1", "top": -138, "left": 243.8, "attrs": { "color": "red" } },
8     {
9       "type": "wokwi-resistor",
10      "id": "r_led",
11      "top": -80,
12      "left": 120,
13      "attrs": { "value": "220" }
14    },
15    {
16      "type": "wokwi-resistor",
17      "id": "r_pull",
18      "top": -20,
19      "left": 150,
20      "attrs": { "value": "10000" }
21    },
22    {
23      "type": "wokwi-pushbutton",
24      "id": "btn1",
25      "top": 54.2,
26      "left": 230.4,
27      "attrs": { "color": "green" }
28    }
29  ],
30  "connections": [
31    [ "esp:D5", "led1:A", "green", [ "h0" ] ],
32    [ "led1:C", "r_led:2", "green", [ "v0" ] ],
33    [ "r_led:1", "esp:GND.1", "green", [ "v0" ] ],
34    [ "r_pull:1", "esp:GND.1", "green", [ "v0" ] ],
35    [ "esp:D4", "btn1:1.1", "green", [ "h0" ] ],
36    [ "btn1:1.1", "r_pull:2", "green", [ "h0" ] ],
37    [ "btn1:2.1", "esp:3V3", "green", [ "h0" ] ]
38  ],
39  "dependencies": {}
40 }
```